

Structures, Unions & Memory Layout

Class 3 – Aggregates, Bit Fields, Alignment & Endianness

Jakub Piela

AGH University of Krakow · Faculty of Physics and Applied Computer Science
“KERNEL” CS Student Research Club · kernel.fis.agh.edu.pl

Academic Year 2025/2026



Educational material prepared for the “KERNEL” CS Student Research Club, AGH University of Krakow. Provided *as is*, for learning purposes only; the author assumes no responsibility for its use or for any errors herein. All third-party names, trademarks, and book covers belong to their respective owners.

Outline

- 1 Structures & Padding
- 2 Alignment
- 3 Flexible Array Members
- 4 Unions & Type Punning
- 5 Enumerations & Bit Fields
- 6 Endianness

Structures: grouping related data

A `struct` bundles members that may have different types into a single object with a contiguous layout:

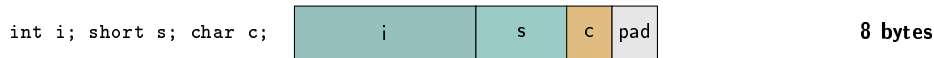
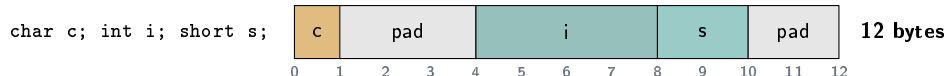
```
1 struct Point { int x, y; };           // a named structure type
2 struct Point p = { .x = 3, .y = 4 }; // designated initialiser
3 int dx = p.x;                         // member access with .
4 struct Point* pp = &p;
5 int dy = pp->y;                       // through a pointer with ->
```

Tip

Prefer *designated* initialisers (`.x = 3`): they are order-independent and self-documenting, and any omitted member is zero-initialised.

Members are laid out with padding

The compiler inserts *padding* so each member meets its alignment. Member order therefore changes the size of the object.¹



Same members, different order: **12 B** vs **8 B**. Order members from largest to smallest alignment to minimise padding.

¹Source: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §6.3.

Measuring layout: sizeof, alignof, offsetof

```
1 #include <stddef.h>      // offsetof
2 #include <stdalign.h>    // alignof (C11; a keyword in C23)
3
4 struct S { char c; int i; short s; };
5
6 sizeof(struct S);         // 12 -- total size, padding included
7 alignof(struct S);       // 4  -- the widest member's alignment
8 offsetof(struct S, i);    // 4  -- byte offset of member i
```

Note

sizeof always counts trailing padding, so an array of structs stays correctly aligned. offsetof is the portable way to learn a member's position — never hard-code it.

Packing: trading speed for size

`__attribute__((packed))` removes all padding — useful for wire formats, dangerous for everyday data:

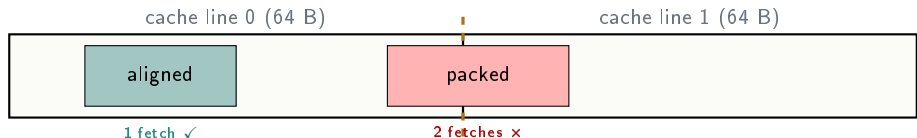
```
1 struct __attribute__((packed)) Raw {  
2     char c;      // offset 0  
3     int i;       // offset 1 -- now MISALIGNED  
4     short s;     // offset 5  
5 };              // sizeof == 7, alignof == 1
```

Warning

A packed struct can place an `int` on an odd address. On strict architectures that faults (next section); on x86 it merely makes every access slower and can straddle a cache line.

Why misalignment hurts: cache-line straddle

A naturally aligned object lives inside one cache line — one fetch. A packed object can cross the 64-byte boundary, forcing the CPU to read *two*.



Pack only data that truly needs a fixed byte layout; keep hot in-memory structures naturally aligned.

Controlling alignment: `alignas` & `alignof`

`alignof` queries a type's alignment; `alignas` strengthens an object's alignment (e.g. to a cache line or SIMD boundary):

```
1 #include <stdalign.h>
2 alignas(64) char line_buf[64];      // start on a 64-byte boundary
3 alignas(int) char storage[sizeof(int)]; // align as another type
4
5 static_assert(alignof(max_align_t) >= 8, "weird ABI"); // C23 keyword
```

From *Modern C*

Alignment is part of C's memory model: every type has a required alignment, and accessing an object through a misaligned pointer is undefined behaviour.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §12.7.

Aligned heap memory: `aligned_alloc`

`malloc` returns memory aligned for *any* ordinary type. When you need stricter alignment, ask for it explicitly:

```
1 // size MUST be a multiple of the alignment (C11 requirement)
2 float* v = aligned_alloc(32, 32 * sizeof *v); // 32-byte aligned (AVX)
3 if (!v) { /* handle failure */ }
4 /* ... use v ... */
5 free(v); // freed normally
```

Note

Ordinary `malloc` already satisfies `max_align_t`; reach for `aligned_alloc` only for over-aligned needs such as SIMD vectors or DMA buffers.

When alignment is violated: the bus error

On architectures that require aligned access, a misaligned load does not run slower — it *traps*:

```
1 char buf[8];  
2 int* p = (int*)(buf + 1);    // deliberately misaligned  
3 int x = *p;                  // SIGBUS on ARM/SPARC; UB everywhere
```

Warning

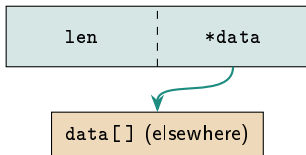
A misaligned access is **undefined behaviour**. It may yield SIGBUS (a “bus error”), silently corrupt data, or appear to work until the compiler vectorises the loop. Never cast a `char*` to a wider type unless you know it is aligned.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §12.7.

The problem: a header plus a variable-length tail

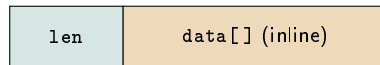
A common need: a length followed by that many bytes. The naive layout uses *two* allocations and a pointer chase; a flexible array member packs both into *one* contiguous block.

Two allocations



2 mallocs, pointer chase

One block (FAM)



1 malloc, contiguous ✓

Flexible array members (C99/C23)

A trailing `[]` array has no size of its own — allocate the header *plus* the elements as one block:

```
1 struct Str { size_t len; char data[]; }; // data[] = flexible member
2 struct Str* make(const char* s) {
3     size_t n = strlen(s);
4     struct Str* p = malloc(sizeof *p + n + 1); // header + n+1 bytes
5     if (!p) return nullptr;
6     p->len = n;
7     memcpy(p->data, s, n + 1);
8     return p; // one free() releases all
9 }
```

From *Modern C*

Allocate enough storage for the intended elements, and keep a length member in step with the array.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §13.1.3 (flexible array members).

Unions: one storage, many views

All members of a union share the same memory; its size is that of the largest member:

```
1 union Value { int i; float f; void* p; };
2
3 union Value v;
4 v.i = 42;           // active member is now 'i'
5 float g = v.f;      // reads the bytes of 42 AS a float -- type punning
```

storage:

i / f / p all overlap here

Type punning & the effective-type rule

Reading a member you did not write is allowed for unions, but the bytes are *reinterpreted*, not converted:

```
1 union { uint32_t u; float f; } pun = { .f = 1.0f };
2 printf("%08x\n", pun.u);      // 3f800000 -- IEEE-754 bits of 1.0f
```

From *Modern C*

Objects must normally be accessed through their *effective type*, but **any** member of an object of union type may be read — this is how C sanctions type punning.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §12.6, Takeaways 12.6 #1–#2.

Warning

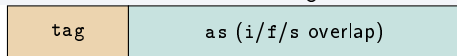
Type punning via pointer casts (not a union) usually breaks the aliasing rules. Use a `union` or `memcpy` instead.

Tagged unions: remember the active member

A union alone cannot tell you which member is valid. Pair it with a *tag* and you have a safe sum type:

```
1 enum Kind { K_INT, K_FLT, K_STR };
2 struct Variant {
3     enum Kind tag;
4     union { long i; double f; char* s; } as;
5 };
6 #define AS_INT(v)  (assert((v).tag == K_INT), (v).as.i)  // checked accessor
```

layout:



Enumerations & C23 fixed underlying type

An enum names a set of related integer constants. C23 lets you fix the underlying type, which pins the ABI and allows forward declarations:

```
1 enum Color { RED, GREEN, BLUE };           // 0, 1, 2 (type chosen by compiler)
2 enum Status : uint8_t { OK = 0, WARN = 1, FAIL = 255 }; // C23: 1 byte exactly
```

From *Modern C*

Before C23 the underlying type of an enumeration was implementation-chosen; C23 lets you specify it, which is essential for stable on-disk and on-wire formats.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §5.6.2, Takeaway 5.6.2 #6.

Bit fields: members measured in bits

Bit fields pack several small values into one integer — handy for flags and hardware registers:

```
1 struct Flags {  
2     unsigned ready : 1;    // 1 bit  
3     unsigned mode  : 2;    // 2 bits  
4     unsigned count : 5;    // 5 bits -> all share one storage unit  
5 };
```

Recommendation

Always give bit fields an *explicit* signedness. A bare `int` bit field may be signed or unsigned depending on the compiler, so a 1-bit `int` field stores only 0 and -1 .^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §6.3.4, Takeaway 6.3.4 #3.

Warning

Allocation *order* within a unit and whether a field may straddle units are implementation-defined — bit-field layout differs across compilers and endianness, so it is unreliable for on-wire formats.

C23 `_BitInt`: types of an exact bit width

Where bit fields are members, `_BitInt(N)` gives standalone integers of a precise width — useful for protocols and fixed-point math:

```
1 unsigned _BitInt(3) u3 = 7uwb;    // holds 0..7
2 signed   _BitInt(12) s12 = -2048wb; // exactly 12 bits, sign-extended
3 _BitInt(128) big = 0;             // wider than any built-in integer
```

Note

`_BitInt(N)` carries no padding bits in arithmetic, so widths compose predictably — unlike bit fields, whose packing is implementation-defined.^a

^aSource: J. Gustedt, *Modern C*, 3rd ed., Manning (2025), §5.7.7.

Putting it together: a compact packet header

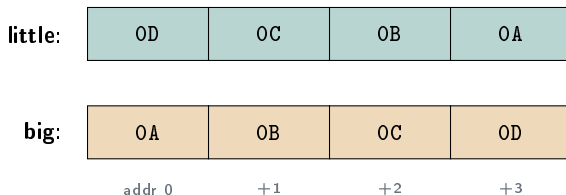
```
1 struct __attribute__((packed)) Packet {  
2     enum : uint8_t { DATA, ACK, NAK } type; // 1 byte, fixed  
3     unsigned version : 4; // 4 bits  
4     unsigned flags : 4; // 4 bits -> 1 byte total  
5     uint16_t length; // payload size  
6     uint8_t payload[]; // flexible array member  
7 };
```

Tip

One declaration combines a fixed-width enum, bit fields, packing, and a flexible array member — every layout tool from this lecture in a single wire-format header.

Byte order: little vs big endian

A multi-byte value can be stored low-byte-first (little endian, x86/ARM) or high-byte-first (big endian, networks). The 32-bit value 0x0A0B0C0D at increasing addresses:



Detecting and converting byte order

Never inspect raw bytes to guess endianness in portable code — convert at the boundary instead:

```
1 #include <arpa/inet.h>           // htonl / ntohl: host <-> network (big endian)
2 uint32_t on_wire = htonl(value);  // serialise
3 uint32_t host    = ntohl(on_wire); // deserialise
4
5 // C23 also offers a compile-time check:
6 if (__STDC_ENDIAN_NATIVE__ == __STDC_ENDIAN_LITTLE__) { /* little endian */ }
```

Note

Keep all internal data in host order and convert only when reading or writing files, sockets, or hardware. “Network byte order” is big endian.

Layout cheat-sheet

Tool	Use it for
<code>struct</code> (+ reorder)	Grouping data; order members to cut padding
<code>alignas</code> / <code>alignof</code>	Querying/forcing alignment (SIMD, cache lines)
<code>aligned_alloc</code>	Over-aligned heap blocks
<code>__attribute__((packed))</code>	Fixed on-wire byte layout (accept slow access)
Flexible array member	Header + variable tail in one allocation
<code>union</code> (+ tag)	Overlapping views / safe sum types
<code>enum</code> : <code>T</code> / bit field / <code>_BitInt</code>	Compact, fixed-width fields
<code>htonl</code> / <code>ntohl</code>	Crossing an endianness boundary

Layout is not an accident: order members, mind alignment, and convert byte order at every boundary.

- J. Gustedt, *Modern C*, 3rd ed., Manning, 2025. ISBN 978-1633437777. Drawn on here:
 - §5.6.2 fixed-type enumerations; §5.7.7 `_BitInt`
 - §6.3 structures; §6.3.4 bit fields
 - §12.6 effective types / unions; §12.7 alignment
 - §13.1.3 flexible array members
- ISO/IEC 9899:2024 (C23) — `<stdalign.h>`, `<stddef.h>`, `__STDC_ENDIAN_NATIVE__`.

Based on the KERNEL class notes by Jakub Piela. © 2026.